ECE 562 Assignment 3: PPO II

Section 2: Experiments and Report

Emmanuel Oluwasegun Olateju (eolateju@ualberta.ca)

October 17, 2026

Setup

All experiments use the provided PPO implementation on `Hopper-v5` with the default hyperparameters (rollout length 2048, mini-batch 64, 10 epochs, learning rate 3×10^{-4}, $\gamma = 0.99$, $\lambda = 0.95$, $\epsilon = 0.2$, entropy coefficient 0, value coefficient 0.5, gradient clipping 0.5). Every configuration was trained for 3×10^5 environment steps ($\lfloor 300000/2048 \rfloor = 146$ PPO iterations, 299,008 steps) with seeds $\{1, 2, 3\}$. All 15 runs completed; none triggered the numerical-instability stop. Every curve shows the mean across seeds with a shaded ± 1 standard deviation band against environment steps. The evaluation return is that of the deterministic policy mean over 5 episodes, computed every 5 iterations. The exact commands are in `code/README.md`:

```
JOBS=12 bash run_all.sh      # 15 runs -> runs/*.csv
python plot.py               # -> figs/*.png
```

The diagnostic `kl` is the estimator $\widehat{\mathrm{KL}}(\pi_{\theta_{\text{old}}} \,\|\, \pi_\theta) = \hat{\mathbb{E}}_{(s,a)\sim\pi_{\theta_{\text{old}}}}[(r_\theta - 1) - \log r_\theta]$, computed on all 2048 samples of the rollout used for the update. Here $\pi_{\theta_{\text{old}}}$ is the policy that *collected* that rollout.

config	final eval	last-5 eval	first eval ≥ 1000	`kl_start`	`clipfrac_start`	`kl_end`	entropy
$k = 0$, clip	2106 ± 473	1924 ± 688	92k–102k	0	0	0.013	3.2
$k = 2$, clip	2060 ± 294	1822 ± 327	113k–225k	0.126	0.57	0.014	3.4
$k = 4$, clip	1578 ± 588	1552 ± 405	215k–256k	0.111	0.56	0.013	3.6
$k = 2$, ratio	256 ± 36	220 ± 44	never	11.9	0.97	16.6	-0.9
$k = 2$, naive	-1998 ± 784	-1882 ± 752	never	2.9×10^{12}	1.00	2.3×10^{12}	-17.3

Table 1: Summary over 3 seeds. "Final eval" is the last evaluation and "last-5 eval" the mean of the last five evaluations (mean $\pm$ std across seeds). "First eval ≥ 1000" gives the range over seeds of the first evaluation reaching a return of 1000. The KL, clip-fraction and entropy columns are averages over the second half of training ($> 1.5 \times 10^5$ steps). Entropy is that of the 3-D Gaussian policy.

Q5: How Stale Can PPO's Data Be?

(a) Modification

Implementation.

- **Staleness parameter** (`train.py`). A new flag `-k/--staleness` (default 0) is added. The training loop keeps `history = collections.deque(maxlen=k+1)`. At every PPO iteration i:
  1. A fresh rollout is collected with the current policy π_{θ_i} and appended to `history`, so every run uses the same number of environment steps regardless of k.
  2. The update uses `data = history[0]`. Once the deque is full this is the rollout collected at iteration $i-k$ by $\pi_{\theta_{i-k}}$. Before that (iterations $i \leq k$) it is the oldest stored rollout, as specified. The deque discards older rollouts automatically, so exactly the $k+1$ most recent rollouts are kept. With $k=0$ the code reduces exactly to the original PPO.

- **Value re-evaluation** (`buffer.py`). A new method `RolloutBuffer.refresh_values(ac)` recomputes, under `torch.no_grad()`, $V_\phi(s_t)$ for all 2048 stored states and the bootstrap value $V_\phi(s_T)$ with the *current* critic. It is called right before `compute_gae`. Observations, actions, rewards, done flags and the behavior log-probabilities $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ are not touched, so the denominator of r_θ is the density of the policy that actually generated each action.

- **Diagnostics.** `ratio_stats(ac, data, ...)` is called on the selected stale rollout `data` before (`_start`) and after (`_end`) the 10 epochs, so all logged KL, ratio-percentile and clip-fraction values refer to the batch actually used.

- Additionally, the update is wrapped in a `try` block. If the loss or KL becomes non-finite, or the Gaussian receives NaN parameters, training stops, the rows so far are kept, and a `.unstable` file records the step. This never triggered in our runs.

Why re-evaluate the values? GAE is built from the TD errors $\delta_t = r_t + \gamma V(s_{t+1})(1-d_{t+1}) - V(s_t)$, and the critic targets are $R_t = \hat{A}_t + V(s_t)$. The stored values are $V_{\phi_{i-k}}(s_t)$, the outputs of a critic that has since been trained for k more iterations ($10k$ epochs). Using them would: (i) compute advantages against an outdated baseline. That baseline approximated $V^{\pi_{\theta_{i-k-1}}}$, i.e. it was already one iteration behind when the data were collected, so the advantage bias grows with k. (ii) Regress the current critic towards targets formed from an old critic, undoing k iterations of value learning. The critic is not a property of the data. It is our current best estimate of the value function, and it costs nothing to re-query it on the stored states. So the most accurate available V should be used.

Why must $\log \pi_{\theta_{\text{old}}}(a_t \mid s_t)$ not be recomputed? The ratio $r_\theta = \pi_\theta(a \mid s)/\pi_{\theta_{\text{old}}}(a \mid s)$ is an importance weight. Its denominator must be the density under which the action was actually *sampled*, namely $\pi_{\theta_{i-k}}$. Recomputing it with the current network would replace the denominator by π_{θ_i}. That would give $r_\theta \equiv 1$ at the start of every update, which falsely declares the data on-policy. It would remove the only correction PPO has for the action distribution (Q2a), make `kl_start` and `clipfrac_start` identically zero, and centre the clipping region on a policy that never generated the data. Unlike V, the behavior log-probability is a fixed fact about how the data were generated, not an estimate to be improved.

Is the re-evaluated GAE on-policy for π_{θ_i}? No. Re-evaluation only changes the baseline. The states $s_t \sim d^{\pi_{\theta_{i-k}}}$, the actions $a_t \sim \pi_{\theta_{i-k}}$ and the rewards along the trajectory all come from the old policy. The λ-return inside GAE sums rewards collected by $\pi_{\theta_{i-k}}$ over many future steps, without any per-step correction. So $\hat{A}$ is (at best) an estimate of $A^{\pi_{\theta_{i-k}}}$, not $A^{\pi_{\theta_i}}$. Moreover, the current critic approximates neither exactly, since it is trained on a mixture of stale rollouts. The state-distribution mismatch $w_\theta(s)$ (Q2) is also untouched.

(b) Results

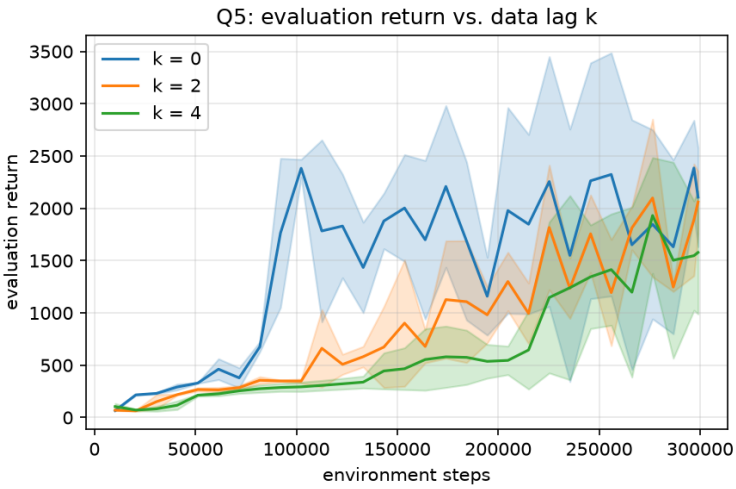
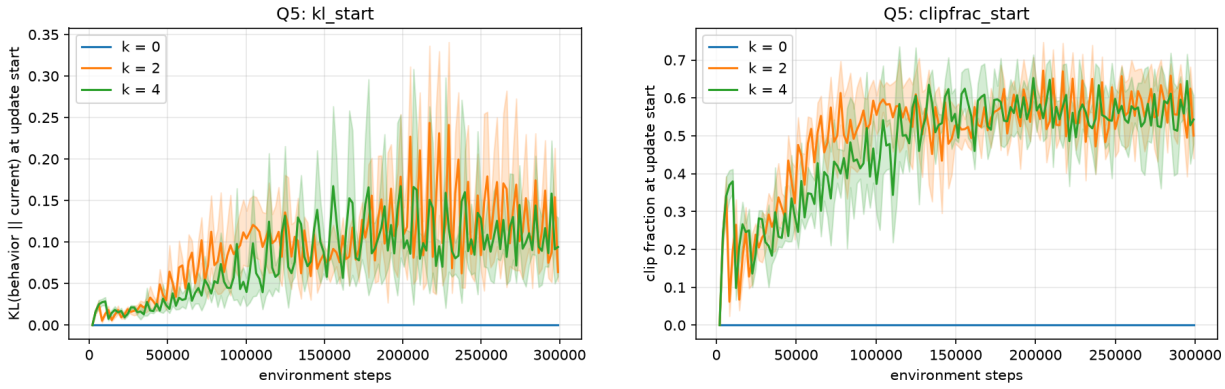


Figure 1: Q5: evaluation return for $k \in \{0, 2, 4\}$ (clip loss).



(a) `kl_start` (b) `clipfrac_start`

Figure 2: Q5: mismatch between the behavior policy and the current policy at the start of each update.

Observations.

- **Return** (Fig. 1, Table 1). Learning is clearly delayed by staleness. With $k = 0$ all seeds pass a return of 1000 at about 1×10^5 steps. With $k = 2$ this happens between 1.1×10^5 and 2.25×10^5, and with $k = 4$ only after 2.1–2.6 $\times 10^5$ steps. At $k = 2$ the final return (2060 ± 294) catches up with $k = 0$ (2106 ± 473), but the area under the curve is much smaller. At $k = 4$ the final return is lower (1578 ± 588) and the curve is still rising.

- **`kl_start` and `clipfrac_start`** (Fig. 2). For $k = 0$ both are exactly zero, a sanity check that the update starts on-policy ($r_\theta \equiv 1$). For $k > 0$ both grow during the first $\sim 1.5 \times 10^5$ steps and then plateau at `kl_start` $\approx$ 0.1–0.13 and `clipfrac_start` $\approx$ 0.55–0.6. In the second half, the

5th/95th percentiles of r_θ at the start of the update are 0.39/1.83 ($k = 2$) and 0.41/1.81 ($k = 4$). So well over half the batch is outside $[1 - \epsilon, 1 + \epsilon]$ *before a single gradient step.*

- **Unexpected result.** We expected $k = 4$ to show a larger start-of-update mismatch than $k = 2$, but the two plateaus overlap (0.111 vs. 0.126). The explanation is in the `_end` diagnostics: `kl_end` ≈ 0.013 for every k, *smaller* than `kl_start`. So, measured against the stale behavior policy, each update moves the policy *back towards* $\pi_{\theta_{i-k}}$ (see (c)). Because every update re-anchors the policy to old data, the total drift between $\pi_{\theta_{i-k}}$ and π_{θ_i} does not grow linearly with k; it saturates. In addition, $k = 4$ learns more slowly, so its policy moves less per iteration, and keeps a larger entropy (3.6 vs. 3.4). A wider Gaussian produces a smaller KL for the same mean shift.

(c) Analysis

How stale can the data be? In this setup, performance already degrades at $k = 2$: it reaches a return of 1000 roughly 1.2–2.4× later than $k = 0$, with larger seed variance early on. The degradation becomes substantial at $k = 4$, which ends lower and is still far from converged at 3×10^5 steps. Only $k = 0$ (truly on-policy) is free of the effect. PPO tolerates a small lag in the sense that it does not diverge, which is clipping at work (Q6). But any lag that makes r_θ leave the trust region at the start of the update costs sample efficiency.

Explanation using Q2. Even with exact advantages, the correctly reweighted gradient needs the full factor $w_\theta(s)\, r_\theta(s, a)$, where $w_\theta(s) = d^{\pi_\theta}(s)/d^{\pi_{\theta_{\text{old}}}}(s)$. PPO uses only r_θ and implicitly sets $w_\theta(s) \approx 1$. The bound $\|d^{\pi_\theta} - d^{\pi_{\theta_{\text{old}}}}\|_1 \leq \frac{2\gamma}{1-\gamma} \mathbb{E}_{s \sim d^{\pi_{\theta_{\text{old}}}}}[D_{TV}]$ says this approximation is only justified while the two policies are close. With $\gamma = 0.99$ the prefactor is 198. With `kl_start` ≈ 0.12, Pinsker gives $D_{TV} \leq \sqrt{\text{KL}/2} \approx 0.24$ on average, so the bound on the state-distribution mismatch is vacuous ($\gg 2$). The data are therefore drawn from a state distribution that may differ substantially from $d^{\pi_{\theta_i}}$, and the gradient puts weight on states the current policy no longer visits and too little on states it does visit. In addition, $\hat{A} \approx A^{\pi_{\theta_{i-k}}} \neq A^{\pi_{\theta_i}}$ (Q3b), a second error that also grows with the policy gap.

Explanation using Q4. w_θ cannot be computed because $d^\pi = (1 - \gamma)(I - \gamma P_\pi^\top)^{-1} p_0$ depends on the unknown transition kernel and, through the matrix inverse, on the policy at *all* states. Changing the policy at one state changes P_π in that row, and the inverse spreads the change to every state reachable from it. In Hopper, slightly different joint torques early in an episode lead to different gaits, heights and fall times, so after k iterations of learning, the states visited by $\pi_{\theta_{i-k}}$ (e.g. falling after a short hop) are systematically different from those of π_{θ_i} (longer hopping). The stale rollout's states are simply the wrong states to improve the current policy on, and nothing in the loss corrects for this.

The clip fraction and the fraction of useful samples. For a sample with ratio r and advantage $\hat{A}$, the clipped objective is $\min(r\hat{A},\ \text{clip}(r, 1 - \epsilon, 1 + \epsilon)\hat{A})$. Its gradient is zero only when the minimum selects the *clipped* (constant) branch. That happens exactly when $r > 1 + \epsilon$ and $\hat{A} > 0$, or when $r < 1 - \epsilon$ and $\hat{A} < 0$, i.e. when the policy has already moved in the direction the advantage asks for. In the other two out-of-range cases, $r > 1 + \epsilon,\ \hat{A} < 0$ and $r < 1 - \epsilon,\ \hat{A} > 0$, the minimum selects the unclipped branch $r\hat{A}$, which still has gradient $\hat{A} \nabla_\theta r$. Every sample with $|r - 1| \leq \epsilon$ (and

$\hat{A} \neq 0$) contributes. So

$$\text{fraction contributing} = (1 - \texttt{clipfrac}) + \Pr\left[|r-1| > \epsilon \text{ and the unclipped branch is active}\right] \;\geq\; 1 - \texttt{clipfrac}.$$

The inequality is strict whenever some out-of-range sample has the "wrong" sign of advantage. This is common with stale data, because $\hat{A}$ is not correlated with the direction the policy moved over the last k iterations (it was estimated for different data). Also, `clipfrac_start` is only a snapshot. During the 10 epochs some samples enter and leave the clipping region.

Our results show why this matters. For $k \in \{2, 4\}$, $1 - \texttt{clipfrac_start} \approx 0.43$, so fewer than half the samples are guaranteed to give a "normal" policy-gradient signal. A large share of the remaining non-zero gradients come from the out-of-range samples whose unclipped branch is active. Their gradient $\hat{A}\nabla r$ with $\hat{A} < 0,\ r > 1 + \epsilon$ (or $\hat{A} > 0,\ r < 1 - \epsilon$) pushes r back towards 1, i.e. *towards the stale behavior policy*. The outcome is the observed drop from $\texttt{kl_start} \approx 0.12$ to $\texttt{kl_end} \approx 0.013$ and from $\texttt{clipfrac} \approx 0.57$ to 0.18. Much of each update is spent pulling the policy back into a trust region centred on an outdated policy, rather than improving the current one. This explains the slower learning in Fig. 1.

Why PPO discards its data after every iteration. Reusing a rollout after the policy has moved has three effects. (1) The number of samples with an informative clipped gradient shrinks ($1 - \texttt{clipfrac}$ falls from 1 to ≈ 0.43). (2) The trust region is centred on the wrong policy, so updates partly undo progress. (3) The uncorrected state-distribution mismatch $w_\theta(s)$ and advantage mismatch $A^{\pi_\theta} \neq A^{\pi_{\theta_{\text{old}}}}$ grow with the policy gap and bias the gradient. Clipping keeps the procedure stable but cannot remove any of these effects. Since a fresh rollout costs only 2048 environment steps and restores $r_\theta \equiv 1$, $w_\theta \equiv 1$ and $A^{\pi_\theta} = A^{\pi_{\theta_{\text{old}}}}$ at the start of the update, PPO stays (almost) on-policy: it reuses each rollout only for the 10 epochs of one iteration, where the policy is kept close by design.

Q6: Does the Importance Ratio Help with Stale Data?

(a) Modification and equality of gradients

Implementation (`ppo.py`). `PPOAgent` takes `loss_type`$\in$ {`naive`, `ratio`, `clip`}, selected with `--loss` in `train.py`. With `ratio = exp(newlogprob - b_logprobs)` and the normalized advantages:

$$\begin{aligned}
\text{naive:} \quad & \texttt{pg_loss} = -\hat{\mathbb{E}}[\log \pi_\theta(a \mid s)\, \hat{A}], \\
\text{ratio:} \quad & \texttt{pg_loss} = -\hat{\mathbb{E}}[r_\theta(s,a)\, \hat{A}], \\
\text{clip:} \quad & \texttt{pg_loss} = -\hat{\mathbb{E}}\big[\min\big(r_\theta \hat{A},\ \text{clip}(r_\theta, 1-\epsilon, 1+\epsilon)\hat{A}\big)\big].
\end{aligned}$$

The value loss, gradient clipping, advantage normalization (once per rollout) and the stored behavior log-probabilities are identical for all three.

Equal gradients at $r_\theta = 1$. $\hat{A}$ and $\log \pi_{\theta_{\text{old}}}(a \mid s)$ do not depend on θ. Since $r_\theta = \exp(\log \pi_\theta - \log \pi_{\theta_{\text{old}}})$,

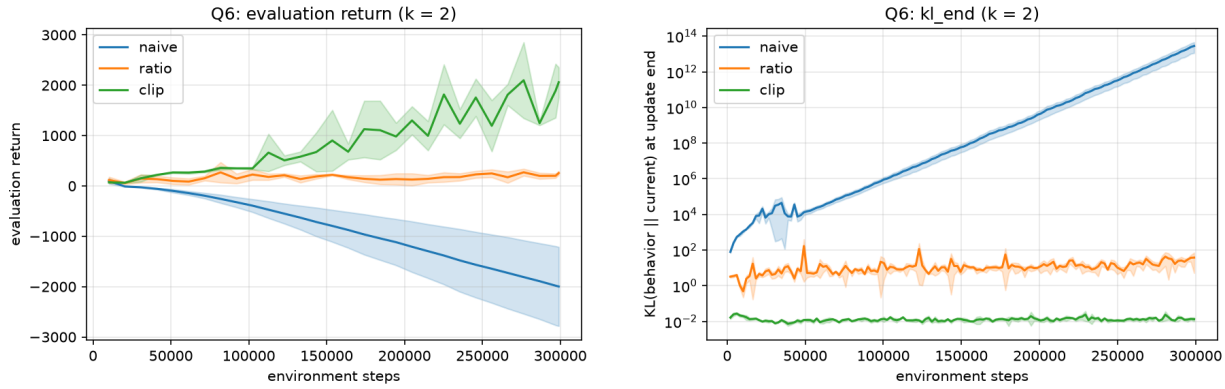
$$\nabla_\theta r_\theta(s,a) = r_\theta(s,a)\, \nabla_\theta \log \pi_\theta(a \mid s).$$

- Naive: $\nabla_\theta L_{\text{naive}} = -\hat{\mathbb{E}}[\hat{A}\, \nabla_\theta \log \pi_\theta]$.

- Ratio: $\nabla_\theta L_{\text{ratio}} = -\hat{\mathbb{E}}[\hat{A}\, r_\theta\, \nabla_\theta \log \pi_\theta]$, which equals the naive gradient when $r_\theta = 1$ for every sample.
- Clip: at $r_\theta = 1$ we have $1-\epsilon < r_\theta < 1+\epsilon$ (for $\epsilon > 0$), so $\text{clip}(r_\theta, 1-\epsilon, 1+\epsilon) = r_\theta$ in a neighbourhood of this point. Both arguments of the min coincide and equal $r_\theta \hat{A}$, so the per-sample objective is locally $r_\theta \hat{A}$ and its gradient is $\hat{A} \nabla_\theta r_\theta = \hat{A}\, r_\theta \nabla_\theta \log \pi_\theta = \hat{A} \nabla_\theta \log \pi_\theta$.

So all three losses have the gradient $-\hat{\mathbb{E}}[\hat{A}\, \nabla_\theta \log \pi_\theta(a \mid s)]$, the standard on-policy policy gradient. This is exactly the situation at the first mini-batch of an update with $k = 0$. The losses differ only once $r_\theta \neq 1$, i.e. for stale data or after the first gradient step.

(b) Results



(a) evaluation return

(b) `kl_end` (log scale)

Figure 3: Q6: the three surrogate losses with $k = 2$. The clip curve is the $k = 2$ run from Q5.

- **clip** learns normally (final 2060 ± 294) with `kl_end` ≈ 0.01–0.02 throughout.
- **ratio** makes almost no progress. The return stays at 100–400 (final 256 ± 36), about the level of a policy that stands and falls. Its `kl_end` is $\sim$ 1–100 (mean 16.6 in the second half), three orders of magnitude above clip, and 97% of samples end the update outside the trust region. The median ratio at the end of an update is 0.01 and the entropy becomes negative ($4.2 \to -2.2$ for seed 1), i.e. the policy collapses.
- **naive** is the worst. The return decreases monotonically from the first evaluation and reaches -1998 ± 784. Its `kl_end` is already ≈ 90 after the first update and then grows *exponentially* (a straight line on the log plot) to $\sim 10^{13}$. The entropy falls to ≈ -17 (-23 for seed 1, i.e. $\log \sigma \approx -9$ per action dimension).
- **Numerical stability.** No run produced a non-finite loss or KL, so all 146 iterations are available for every configuration and no truncation was needed. The naive run is, however, numerically degenerate: its ratios underflow to 0 for essentially all samples (95th percentile $\approx 10^{-17}$) and its value loss is $\sim 10^5$.

(c) Analysis

Without the ratio (naive). The naive loss ignores where the data came from. It treats $(s, a) \sim \pi_{\theta_{\text{old}}}$ as if it were drawn from π_θ, so it lacks *both* corrections of Q3(a): $w_\theta(s)$ and $r_\theta(s, a)$. Even in expectation it is not the gradient of $J(\theta)$ once $\theta \neq \theta_{\text{old}}$. More damaging, it has no term that anchors θ near θ_{old}, and the objective it maximizes, $\hat{\mathbb{E}}[\hat{A} \log \pi_\theta]$, is *unbounded above*. For a Gaussian policy,

$$\hat{A}_i \log \pi_\theta(a_i \mid s_i) = -\hat{A}_i \frac{\|a_i - \mu_\theta(s_i)\|^2_{\Sigma^{-1}}}{2} - \hat{A}_i \sum_j \log \sigma_j + \text{const.}$$

For a sample with $\hat{A}_i < 0$, this term grows without bound as $\mu_\theta(s_i)$ moves away from a_i. Shrinking σ multiplies that growth by $1/\sigma^2$. Positive-advantage samples bound their own term by 0 but cannot balance this. The $\log \sigma$ terms cancel on average because the advantages are normalized to zero mean. Ten epochs of Adam on this objective therefore drive $\sigma \to 0$ (entropy $\to -17$) and push the mean away from the actions that did badly, to arbitrarily large values. So the KL grows geometrically, at a rate set by the learning rate, as observed. Huge action magnitudes explain the *negative* returns: Hopper's control cost $10^{-3}\|a\|^2$ is computed on the raw action, so it dominates the healthy and forward rewards. The failure modes without the ratio are a biased gradient that is not the policy gradient of any nearby policy, combined with an unbounded objective that leads to divergence and entropy collapse. Note that the gradient is never weighted by how likely the sample is under the current policy, so a sample the current policy would never generate still produces a full-size gradient.

With the ratio but without clipping (ratio). Weighting by r_θ restores the action-distribution correction. In expectation, $\hat{\mathbb{E}}[r_\theta \nabla \log \pi_\theta \hat{A}]$ is the Q3(b) estimator, which corrects $a \sim \pi_{\theta_{\text{old}}}$ to $a \sim \pi_\theta$ but still assumes $w_\theta(s) \approx 1$ and $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$. This is why ratio does much better than naive (no divergence, positive return). But:

1. **Unbounded surrogate.** $r\hat{A}$ grows without bound as $r \to \infty$ for $\hat{A} > 0$, and the gain is bounded by 0 as $r \to 0$ for $\hat{A} < 0$. Optimizing it for 10 epochs concentrates probability mass on a few positive-advantage actions. The policy narrows (negative entropy), most stored actions become improbable (median $r_{\text{end}} = 0.01$), and a handful of samples with large r dominate the batch.
2. **High variance.** With r spread over several orders of magnitude, the importance-weighted gradient has a very large variance. Its effective sample size is a small fraction of 2048, so the gradient is noise-dominated, which explains the stalled return.
3. **Approximations break down.** Since `kl` ~ 10, the policies are far apart. Then $w_\theta(s) \approx 1$ and $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$ (Q2, Q3b) are badly violated: the surrogate $\hat{\mathbb{E}}[r\hat{A}]$ is only a first-order local approximation of $J(\theta) - J(\theta_{\text{old}})$, so maximizing it far from θ_{old} no longer improves the true return.

In short, the ratio fixes the bias in the action distribution, but without a constraint the optimizer exploits the surrogate in a region where it is meaningless and the importance weights have huge variance.

How clipping helps. The clipped objective is flat in r once r leaves $[1-\epsilon, 1+\epsilon]$ in the improving direction. This removes the incentive to push any sample's ratio further, which bounds the effective importance weights and the variance of the gradient. It keeps π_θ close to $\pi_{\theta_{\text{old}}}$, where $w_\theta \approx 1$ and $A^{\pi_\theta} \approx A^{\pi_{\theta_{\text{old}}}}$ are reasonable (the bound in Q2b). The pessimistic min also leaves a gradient on samples where the policy moved in the wrong direction, pulling them back. Empirically, `kl_end` ≈ 0.013 for every k, compared with ~ 17 (ratio) and $\sim 10^{12}$ (naive).

What clipping does not fix.

- It does not correct the state distribution: $w_\theta(s)$ is still set to 1 (Q2, Q4).
- It does not correct the advantages: $\hat{A}$ still estimates $A^{\pi_{\theta_{\text{old}}}}$ (with an approximate critic), not A^{π_θ} (Q3b).
- It does not remove the mismatch already present in stale data. The trust region is centred on $\pi_{\theta_{\text{old}}} = \pi_{\theta_{i-k}}$, not on the current policy, so at the start of an update 57% of samples are already clipped and part of the update is spent pulling the policy back towards an outdated policy (Q5c). This is why clip with $k > 0$ is stable but slower than $k = 0$.
- It does not enforce a hard trust region. Clipping only removes the gradient of individual samples; the policy can still move further through the shared network parameters, through other samples, or over many epochs. It gives no guarantee on D_{TV} or KL. That clip stays at `kl` ≈ 0.01 is an empirical property of these hyperparameters (10 epochs, small learning rate, gradient clipping), not a guarantee.

Clipping therefore makes PPO *robust* to mildly off-policy data, but it does not make PPO an off-policy algorithm. The full correction $w_\theta(s)\, r_\theta(s, a)$ with A^{π_θ} is still missing, which is why PPO collects fresh data every iteration.